SANTA CLARA UNIVERSITY

Electrical and Computer Engineering

ECEN 603 *Logic Design Using HDL*

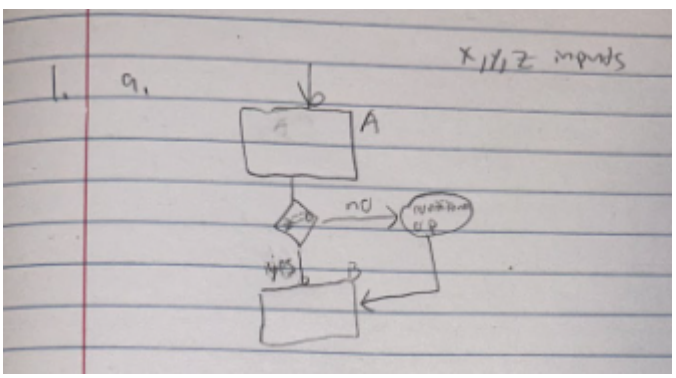
Homework 2

Dylan Thornburg

For each of the following problems, submit the Verilog code for both the design modules and the test bench. Additionally, submit the simulation output.

1. [2 points] Draw ASM charts for each of the following problems. Here, x, y and z are inputs, and A, B, C, and D are the states of a system.
   [3 points] Write the RTL Verilog code for the ASM charts in problem 1.

(a) If z = 0, a transition from state A to state B takes place. If z = 1, a conditional operation takes place followed by a transition from A to B.



SM CODE

```
module sm1(
        input x,y,z,clk,rst,
        output reg r //conditional op
);
        localparam A = 1'b0;
        localparam B = 1'b1;
        reg curr, next;
        always @(posedge clk) begin
                if(rst) curr<=A;
                else curr <=next;
        end
```

```
        always@(*) begin
                r=1'b0; //def
                case(curr)
                        A: begin
                                if(z) r=1'b1;
                                next = B;
                        end

                        B: next = B;

                        default: next = B;
                endcase
        end
endmodule
```

TESTBENCH

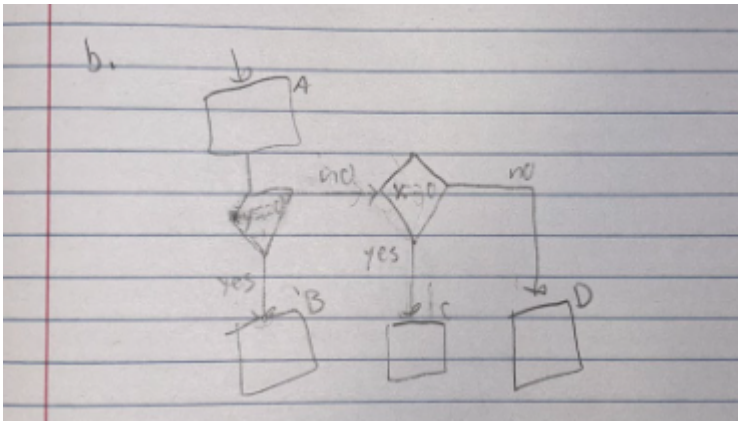
```
module testbench();
   reg x, y, z, clk, rst;
   wire r;

   sm1 dut(
      .x(x),
      .y(y),
      .z(z),
      .clk(clk),
      .rst(rst),
      .r(r)
   );

   initial begin
      clk = 1'b0;
      forever #5 clk = ~clk;
   end

   initial begin
      $display("Start of Problem 1(a) SM1 test.");
      $monitor("t=%0d | clk=%d rst=%d x=%d y=%d z=%d | r=%d", $time, clk, rst, x, y, z, r);

      x = 1'b0;
      y = 1'b0;
      z = 1'b0;

      rst = 1'b1;
      #20;
      rst = 1'b0;

      // Hold z=0 for a couple cycles
      z = 1'b0;
      #30;

      // Z high for one cycle
      z = 1'b1;
      #8;
```

```
    z = 1'b0;
    #12;

    // Hold z=1 spanning a posedge
    z = 1'b1;
    #20;
    z = 1'b0;
    #20;


    $display("End of SM1 test.");
    $finish;
  end
endmodule
```

SIM RESULTS

```
Start of Problem 1(a) SM1 test.
t=0 | clk=0 rst=1 x=0 y=0 z=0 | r=0
t=5 | clk=1 rst=1 x=0 y=0 z=0 | r=0
t=10 | clk=0 rst=1 x=0 y=0 z=0 | r=0
t=15 | clk=1 rst=1 x=0 y=0 z=0 | r=0
t=20 | clk=0 rst=0 x=0 y=0 z=0 | r=0
t=25 | clk=1 rst=0 x=0 y=0 z=0 | r=0
t=30 | clk=0 rst=0 x=0 y=0 z=0 | r=0
t=35 | clk=1 rst=0 x=0 y=0 z=0 | r=0
t=40 | clk=0 rst=0 x=0 y=0 z=0 | r=0
t=45 | clk=1 rst=0 x=0 y=0 z=0 | r=0
t=50 | clk=0 rst=0 x=0 y=0 z=1 | r=0
t=55 | clk=1 rst=0 x=0 y=0 z=1 | r=0
t=58 | clk=1 rst=0 x=0 y=0 z=0 | r=0
t=60 | clk=0 rst=0 x=0 y=0 z=0 | r=0
t=65 | clk=1 rst=0 x=0 y=0 z=0 | r=0
t=70 | clk=0 rst=0 x=0 y=0 z=1 | r=0
t=75 | clk=1 rst=0 x=0 y=0 z=1 | r=0
t=80 | clk=0 rst=0 x=0 y=0 z=1 | r=0
t=85 | clk=1 rst=0 x=0 y=0 z=1 | r=0
t=90 | clk=0 rst=0 x=0 y=0 z=0 | r=0
t=95 | clk=1 rst=0 x=0 y=0 z=0 | r=0
t=100 | clk=0 rst=0 x=0 y=0 z=0 | r=0
t=105 | clk=1 rst=0 x=0 y=0 z=0 | r=0
End of SM1 test.
```

(b) The system starts in state A. If xy = 00, go to B, xy = 01, go to C, xy = 11 go to D; otherwise, go to B.



SM CODE

```
module sm2(
        input x,y,z,clk,rst,
        output reg [1:0] r //represent state
);
        localparam [1:0] A = 2'b00;
        localparam [1:0] B = 2'b01;
        localparam [1:0] C = 2'b10;
        localparam [1:0] D = 2'b11;
        reg [1:0] curr, next;

        always @(posedge clk) begin
                if(rst) curr<=A;
                else curr <=next;
        end

        always@(*) begin
                r = curr;
                case(curr)
                        A: begin
                                if(y==1'b0) next=B;
                                else if (y==1'b1) begin
                                        if(x==1'b0) next = C;
                                        else next=D;
                                end
                        end
                        B: next=B;
                        C: next=C;
                        D: next=D;
                        default: next = A;
                endcase
        end
endmodule
```

TESTBENCH

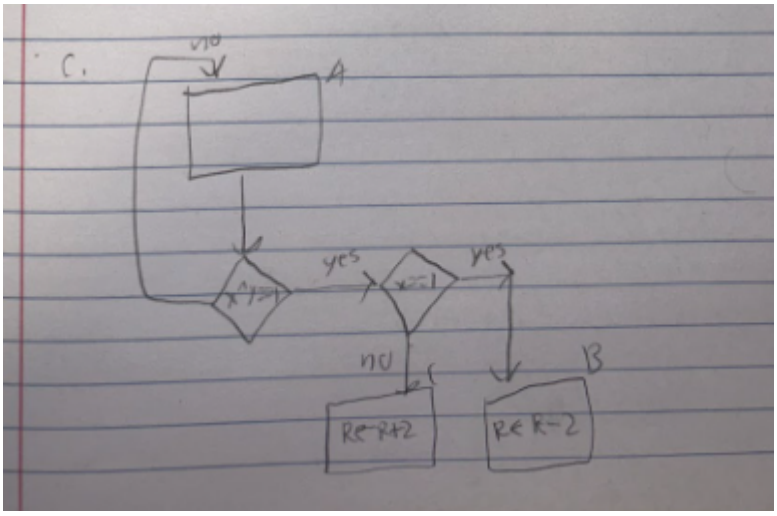
```
module testbench();
   reg x, y, z, clk, rst;
   wire [1:0] r;

   sm2 dut(
      .x(x),
      .y(y),
      .z(z),
      .clk(clk),
      .rst(rst),
      .r(r)
   );

   initial begin
      clk = 1'b0;
      forever #5 clk = ~clk;
   end

   initial begin
      $display("Start of SM2 test");

      $monitor("t=%0d | clk=%d rst=%d x=%d y=%d z=%d | r=%d", $time, clk, rst, x, y, z, r);

      x = 1'b0;
      y = 1'b0;
      z = 1'b0;

      rst = 1'b1;
      #12;
      rst = 1'b0;

      x = 1'b0; y = 1'b0;
      #20;

      rst = 1'b1;
      #10;
      x = 1'b0; y = 1'b1;
      rst = 1'b0;
      #20;

      rst = 1'b1;
      #10;
      x = 1'b1; y = 1'b1;
      rst = 1'b0;
      #20;

      rst = 1'b1;
      #10;
      x = 1'b1; y = 1'b0;
      rst = 1'b0;
      #20;

      $display("End of SM2 test.");
      $finish;
```

```
  end
endmodule
```

SIM RESULTS

```
[dthornburg@linux402604 P1]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Ful
  Feb  8 03:01 2026
Start of SM2 test
t=0 | clk=0 rst=1 x=0 y=0 z=0 | r=x
t=5 | clk=1 rst=1 x=0 y=0 z=0 | r=0
t=10 | clk=0 rst=1 x=0 y=0 z=0 | r=0
t=12 | clk=0 rst=0 x=0 y=0 z=0 | r=0
t=15 | clk=1 rst=0 x=0 y=0 z=0 | r=1
t=20 | clk=0 rst=0 x=0 y=0 z=0 | r=1
t=25 | clk=1 rst=0 x=0 y=0 z=0 | r=1
t=30 | clk=0 rst=0 x=0 y=0 z=0 | r=1
t=32 | clk=0 rst=1 x=0 y=0 z=0 | r=1
t=35 | clk=1 rst=1 x=0 y=0 z=0 | r=0
t=40 | clk=0 rst=1 x=0 y=0 z=0 | r=0
t=42 | clk=0 rst=0 x=0 y=1 z=0 | r=0
t=45 | clk=1 rst=0 x=0 y=1 z=0 | r=2
t=50 | clk=0 rst=0 x=0 y=1 z=0 | r=2
t=55 | clk=1 rst=0 x=0 y=1 z=0 | r=2
t=60 | clk=0 rst=0 x=0 y=1 z=0 | r=2
t=62 | clk=0 rst=1 x=0 y=1 z=0 | r=2
t=65 | clk=1 rst=1 x=0 y=1 z=0 | r=0
t=70 | clk=0 rst=1 x=0 y=1 z=0 | r=0
t=72 | clk=0 rst=0 x=1 y=1 z=0 | r=0
t=75 | clk=1 rst=0 x=1 y=1 z=0 | r=3
t=80 | clk=0 rst=0 x=1 y=1 z=0 | r=3
t=85 | clk=1 rst=0 x=1 y=1 z=0 | r=3
t=90 | clk=0 rst=0 x=1 y=1 z=0 | r=3
t=92 | clk=0 rst=1 x=1 y=1 z=0 | r=3
t=95 | clk=1 rst=1 x=1 y=1 z=0 | r=0
t=100 | clk=0 rst=1 x=1 y=1 z=0 | r=0
t=102 | clk=0 rst=0 x=1 y=0 z=0 | r=0
t=105 | clk=1 rst=0 x=1 y=0 z=0 | r=1
t=110 | clk=0 rst=0 x=1 y=0 z=0 | r=1
t=115 | clk=1 rst=0 x=1 y=0 z=0 | r=1
t=120 | clk=0 rst=0 x=1 y=0 z=0 | r=1
End of SM2 test.
$finish called from file "tb2.v", line 54.
$finish at simulation time                 122
           V C S   S i m u l a t i o n   R e p o r t
Time: 122
CPU Time:      0.190 seconds;       Data structure size:   0.0Mb
Sun Feb  8 03:01:37 2026
[dthornburg@linux402604 P1]$
```

(c) The system starts in state A. If xy = 10, register R is decremented by two and the system transitions to state B. If xy = 01, register R is incremented by two and the control goes to state C. Otherwise, control stays in the same state.



SM CODE

```
module sm3(
        input x,y,z,clk,rst,
        output reg [7:0] r //counter
);
        localparam A = 2'b00;
        localparam B = 2'b01;
        localparam C = 2'b10;
        reg [1:0] curr, next;
        always @(posedge clk) begin
                if(rst) begin
                        curr<=A;
                        r<=8'b0;
                end
                else curr <=next;
                case(curr)
                        A: ;
                        B: r=r-8'd2;
                        C: r=r+8'd2;
                        default: ;
                endcase
        end

        always@(*) begin
                case(curr)
                        A: begin
                                if(x^y) begin
                                        if(x==1'b1) next = B;
                                        else next = C;
                                end
```

```
                                        else next = A;
                                end

                                B: next = B;

                                C: next = C;

                                default: next = A;
                        endcase
            end
endmodule
```

TESTBENCH

```
module testbench();
   reg x, y, z, clk, rst;
   wire [7:0] r;

   sm3 dut(
      .x(x),
      .y(y),
      .z(z),
      .clk(clk),
      .rst(rst),
      .r(r)
   );

   initial begin
      clk = 1'b0;
      forever #5 clk = ~clk;
   end

   initial begin
            $display("Start of SM3 test");

      $monitor("t=%0d | clk=%d rst=%d x=%d y=%d z=%d | r=%d", $time, clk, rst, x, y, z, r);

      x = 1'b0;
      y = 1'b0;
      z = 1'b0;

      rst = 1'b1;
      #12;
      rst = 1'b0;

      x = 1'b0; y = 1'b0;
      #20;

      rst = 1'b1;
      #10;
      x = 1'b1; y = 1'b0;
      rst = 1'b0;
      #40;
```

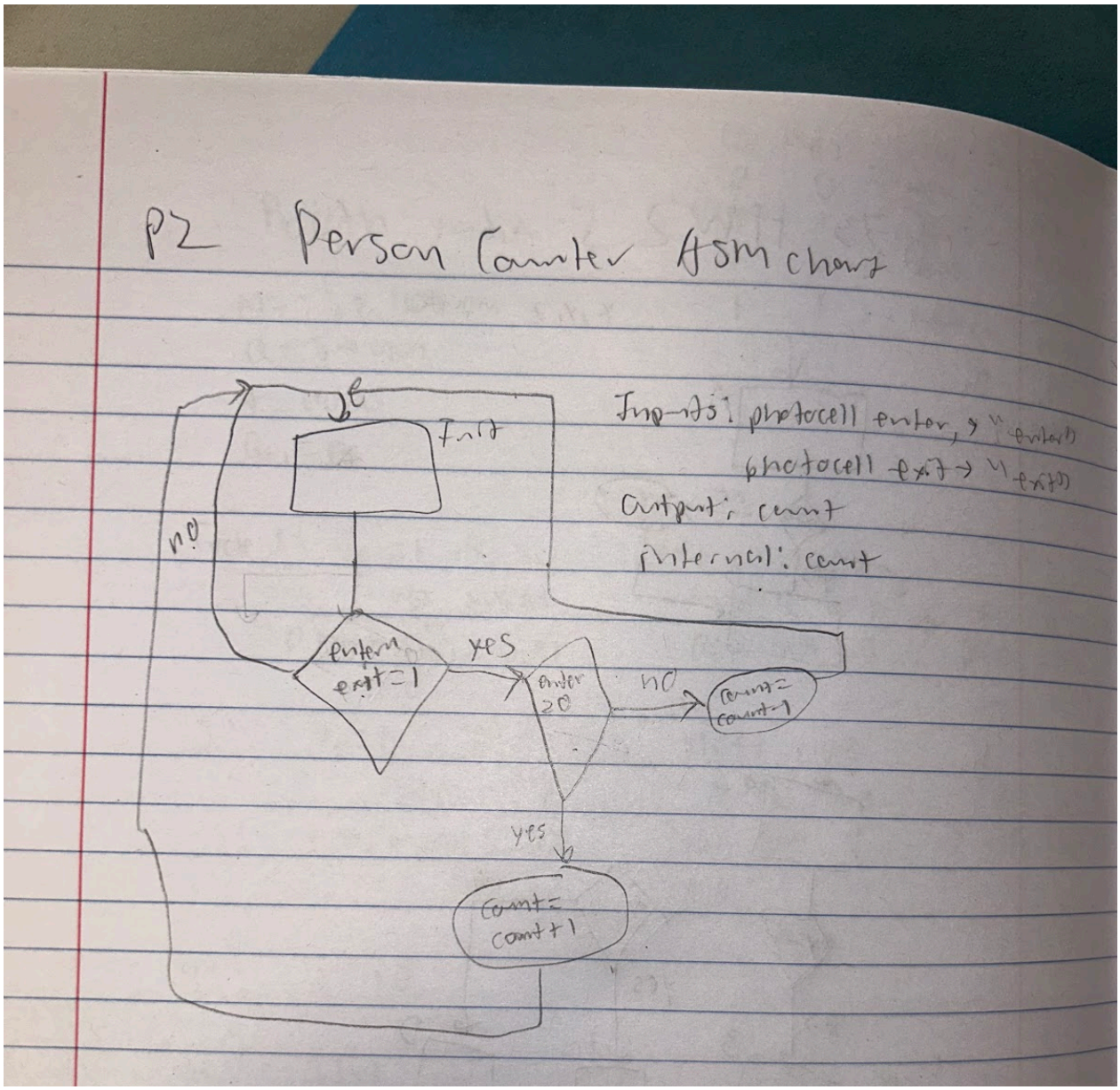
```
        rst = 1'b1;
        #10;
        x = 1'b0; y = 1'b1;
        rst = 1'b0;
        #40;

        rst = 1'b1;
        #10;
        x = 1'b1; y = 1'b1;
        rst = 1'b0;
        #20;

        $display("End of SM3 test.");
        $finish;
    end
endmodule
```

SIM RESULTS

```
Start of SM3 test
t=0 | clk=0 rst=1 x=0 y=0 z=0 | r=  x
t=5 | clk=1 rst=1 x=0 y=0 z=0 | r=  0
t=10 | clk=0 rst=1 x=0 y=0 z=0 | r=  0
t=12 | clk=0 rst=0 x=0 y=0 z=0 | r=  0
t=15 | clk=1 rst=0 x=0 y=0 z=0 | r=  0
t=20 | clk=0 rst=0 x=0 y=0 z=0 | r=  0
t=25 | clk=1 rst=0 x=0 y=0 z=0 | r=  0
t=30 | clk=0 rst=0 x=0 y=0 z=0 | r=  0
t=32 | clk=0 rst=1 x=0 y=0 z=0 | r=  0
t=35 | clk=1 rst=1 x=0 y=0 z=0 | r=  0
t=40 | clk=0 rst=1 x=0 y=0 z=0 | r=  0
t=42 | clk=0 rst=0 x=1 y=0 z=0 | r=  0
t=45 | clk=1 rst=0 x=1 y=0 z=0 | r=  0
t=50 | clk=0 rst=0 x=1 y=0 z=0 | r=  0
t=55 | clk=1 rst=0 x=1 y=0 z=0 | r=254
t=60 | clk=0 rst=0 x=1 y=0 z=0 | r=254
t=65 | clk=1 rst=0 x=1 y=0 z=0 | r=252
t=70 | clk=0 rst=0 x=1 y=0 z=0 | r=252
t=75 | clk=1 rst=0 x=1 y=0 z=0 | r=250
t=80 | clk=0 rst=0 x=1 y=0 z=0 | r=250
t=82 | clk=0 rst=1 x=1 y=0 z=0 | r=250
t=85 | clk=1 rst=1 x=1 y=0 z=0 | r=  0
t=90 | clk=0 rst=1 x=1 y=0 z=0 | r=  0
t=92 | clk=0 rst=0 x=0 y=1 z=0 | r=  0
t=95 | clk=1 rst=0 x=0 y=1 z=0 | r=  0
t=100 | clk=0 rst=0 x=0 y=1 z=0 | r=  0
t=105 | clk=1 rst=0 x=0 y=1 z=0 | r=  2
t=110 | clk=0 rst=0 x=0 y=1 z=0 | r=  2
t=115 | clk=1 rst=0 x=0 y=1 z=0 | r=  4
t=120 | clk=0 rst=0 x=0 y=1 z=0 | r=  4
t=125 | clk=1 rst=0 x=0 y=1 z=0 | r=  6
t=130 | clk=0 rst=0 x=0 y=1 z=0 | r=  6
t=132 | clk=0 rst=1 x=0 y=1 z=0 | r=  6
t=135 | clk=1 rst=1 x=0 y=1 z=0 | r=  0
t=140 | clk=0 rst=1 x=0 y=1 z=0 | r=  0
t=142 | clk=0 rst=0 x=1 y=1 z=0 | r=  0
t=145 | clk=1 rst=0 x=1 y=1 z=0 | r=  0
t=150 | clk=0 rst=0 x=1 y=1 z=0 | r=  0
t=155 | clk=1 rst=0 x=1 y=1 z=0 | r=  0
t=160 | clk=0 rst=0 x=1 y=1 z=0 | r=  0
End of SM3 test.
$finish called from file "tb3.v", line 54.
$finish at simulation time                  162
```

3. [7 points] Design a system to count the number of people in a room. There is one entrance and one exit in the room. The entrance has a photocell that changes a signal from 1 to 0 when the light is interrupted as a person enters the room. The exit has a similar photocell that changes a signal from 1 to 0 when the light is interrupted due to a person leaving the room. The circuit has a counter to count the number of people in the room.

(a) Draw an ASM chart for this system.



(b) Write the RTL Verilog code for this system

SM CODE

```
//assume you cant leave room with 0 people in it
//assume 255 people max
//need to check falling edges of entering and leaving, not merely when low (otherwise you count too much)
module person_counter(
	input enter, exit, clk, rst,
	output reg [7:0] count
);
	reg enter_def, exit_def, enter_falling, exit_falling;

	always @(posedge clk) begin
		if (rst) begin
			count <= 8'b0;
			enter_def<=1;
			exit_def<=1;
		end
		else begin
			enter_def <=enter;
			exit_def <= exit;
		end

		if(enter_falling ^ exit_falling) begin
			if(enter_falling) count <= count + 8'b1;
			else count <= count -8'b1;
		end
	end
	always@(*) begin
		enter_falling = enter_def & ~enter;
		exit_falling = exit_def & ~exit;
	end
endmodule
```

TESTBENCH

```
module testbench();
   reg enter, exit, clk, rst;
   wire [7:0] count;

   person_counter dut(
      .enter(enter),
      .exit(exit),
      .clk(clk),
      .rst(rst),
      .count(count)
   );

   initial begin
      clk = 1'b0;
```

```
    forever #5 clk = ~clk;
end

initial begin
    $display("Start of person counter test");
    $monitor("t=%0d | clk=%d rst=%d enter=%d exit=%d | count=%d", $time, clk, rst, enter, exit, count);

    rst = 1'b1;
    enter = 1'b1;
    exit  = 1'b1;

    #12;
    rst = 1'b0;



        $display("One enters \n");
    enter = 1'b0;
        #10;
    enter = 1'b1;
        #20;
        $display("\n");



        $display("One enters again but takes a longer time \n");
    enter = 1'b0;
        #30;
    enter = 1'b1;
        #20;
        $display("\n");



        $display("One exits. \n");
    exit = 1'b0;
        #10;
    exit = 1'b1;
        #20;
        $display("\n");



        $display("One enters \n");
    enter = 1'b0;
        #10;
    enter = 1'b1;
        #20;
        $display("\n");



        $display("One exits for a while. \n");
    exit = 1'b0;
```

```
        #30;
    exit = 1'b1;
        #20;
        $display("\n");



        $display("One leaves and another comes in. \n");
    enter = 1'b0; exit = 1'b0;
        #10;
    enter = 1'b1; exit = 1'b1;
        #20;
        $display("\n");



    $display("End of person counter test.");
    $finish;
  end
endmodule
```

SIM RESULTS

Because it's a long output, Im just going to copy and paste rather than screenshot. I'll make it match the format though:

```
[dthornburg@linux402604 P2]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Feb  9 01:51 2026
Start of person counter test
t=0 | clk=0 rst=1 enter=1 exit=1 | count=  x
t=5 | clk=1 rst=1 enter=1 exit=1 | count=  0
t=10 | clk=0 rst=1 enter=1 exit=1 | count=  0
One enters

t=12 | clk=0 rst=0 enter=0 exit=1 | count=  0
t=15 | clk=1 rst=0 enter=0 exit=1 | count=  1
t=20 | clk=0 rst=0 enter=0 exit=1 | count=  1
t=22 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=25 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=30 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=35 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=40 | clk=0 rst=0 enter=1 exit=1 | count=  1


One enters again but takes a longer time
```

```
t=42 | clk=0 rst=0 enter=0 exit=1 | count=  1
t=45 | clk=1 rst=0 enter=0 exit=1 | count=  2
t=50 | clk=0 rst=0 enter=0 exit=1 | count=  2
t=55 | clk=1 rst=0 enter=0 exit=1 | count=  2
t=60 | clk=0 rst=0 enter=0 exit=1 | count=  2
t=65 | clk=1 rst=0 enter=0 exit=1 | count=  2
t=70 | clk=0 rst=0 enter=0 exit=1 | count=  2
t=72 | clk=0 rst=0 enter=1 exit=1 | count=  2
t=75 | clk=1 rst=0 enter=1 exit=1 | count=  2
t=80 | clk=0 rst=0 enter=1 exit=1 | count=  2
t=85 | clk=1 rst=0 enter=1 exit=1 | count=  2
t=90 | clk=0 rst=0 enter=1 exit=1 | count=  2
```

One exits.

```
t=92 | clk=0 rst=0 enter=1 exit=0 | count=  2
t=95 | clk=1 rst=0 enter=1 exit=0 | count=  1
t=100 | clk=0 rst=0 enter=1 exit=0 | count=  1
t=102 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=105 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=110 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=115 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=120 | clk=0 rst=0 enter=1 exit=1 | count=  1
```

One enters

```
t=122 | clk=0 rst=0 enter=0 exit=1 | count=  1
t=125 | clk=1 rst=0 enter=0 exit=1 | count=  2
t=130 | clk=0 rst=0 enter=0 exit=1 | count=  2
t=132 | clk=0 rst=0 enter=1 exit=1 | count=  2
t=135 | clk=1 rst=0 enter=1 exit=1 | count=  2
t=140 | clk=0 rst=0 enter=1 exit=1 | count=  2
t=145 | clk=1 rst=0 enter=1 exit=1 | count=  2
t=150 | clk=0 rst=0 enter=1 exit=1 | count=  2
```

One exits for a while.

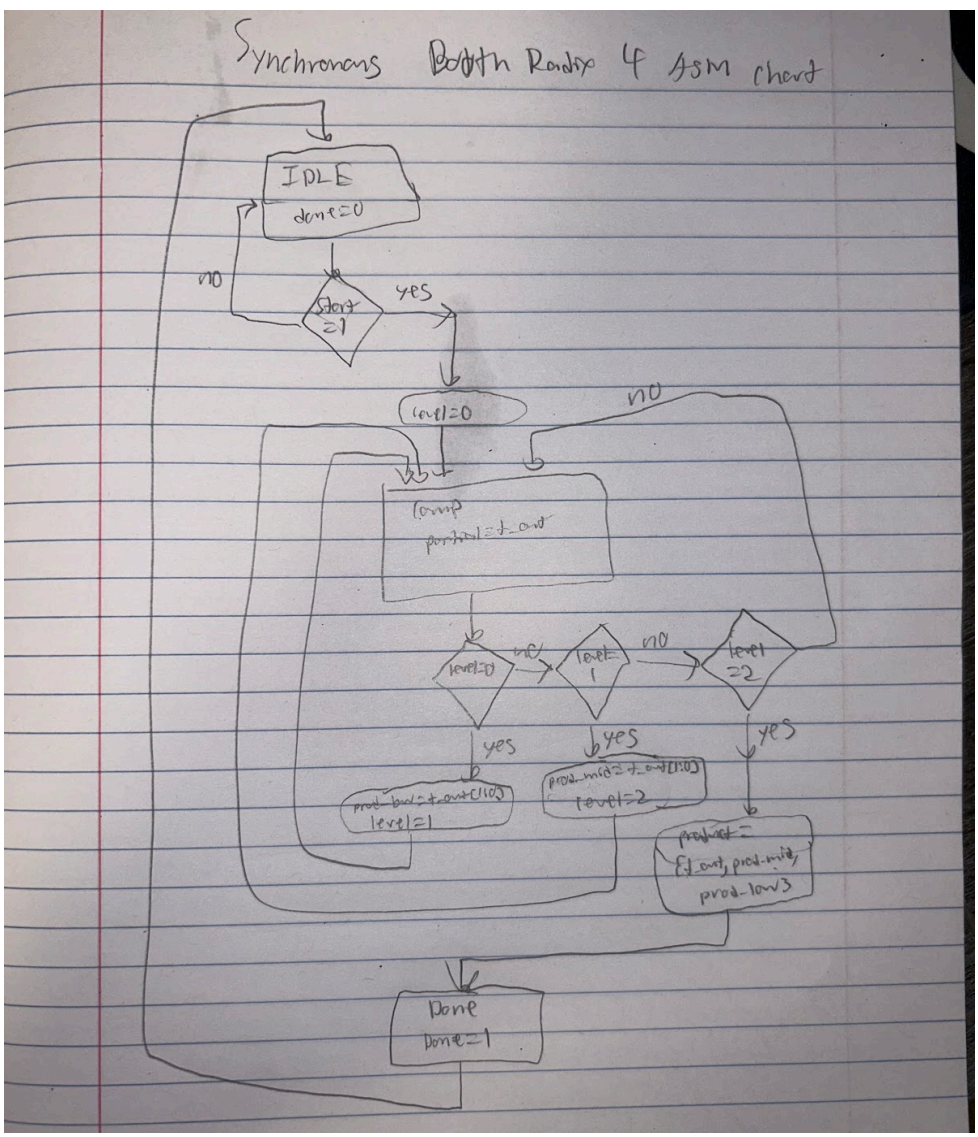
```
t=152 | clk=0 rst=0 enter=1 exit=0 | count=  2
t=155 | clk=1 rst=0 enter=1 exit=0 | count=  1
t=160 | clk=0 rst=0 enter=1 exit=0 | count=  1
t=165 | clk=1 rst=0 enter=1 exit=0 | count=  1
t=170 | clk=0 rst=0 enter=1 exit=0 | count=  1
```

```
t=175 | clk=1 rst=0 enter=1 exit=0 | count=  1
t=180 | clk=0 rst=0 enter=1 exit=0 | count=  1
t=182 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=185 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=190 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=195 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=200 | clk=0 rst=0 enter=1 exit=1 | count=  1


One leaves and another comes in.

t=202 | clk=0 rst=0 enter=0 exit=0 | count=  1
t=205 | clk=1 rst=0 enter=0 exit=0 | count=  1
t=210 | clk=0 rst=0 enter=0 exit=0 | count=  1
t=212 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=215 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=220 | clk=0 rst=0 enter=1 exit=1 | count=  1
t=225 | clk=1 rst=0 enter=1 exit=1 | count=  1
t=230 | clk=0 rst=0 enter=1 exit=1 | count=  1


End of person counter test.
$finish called from file "tb.v", line 86.
$finish at simulation time                 232
           V C S   S i m u l a t i o n   R e p o r t
Time: 232
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Mon Feb  9 01:51:40 2026
[dthornburg@linux402604 P2]$
```

4. [8 points] The operation of the Booth radix-4 multiplier is described by the following table:

x_i	x_{i-1}	x_{i-2}	h	s	d	operation
0	0	0	0	×	×	0
0	0	1	1	0	0	+A
0	1	0	1	0	0	+A
0	1	1	1	1	0	+2A
1	0	0	1	1	1	-2A
1	0	1	1	0	1	-A
1	1	0	1	0	1	-A
1	1	1	0	×	×	0

(a) Draw an ASM chart to implement the operation of a synchronous Booth radix-4 multiplier.



(b) Write the RTL Verilog code for the multiplier.

SM CODE

```
module Booth_Mul_4(
        input clk, rst, start,
        input [5:0] X, A,
        output reg [9:0] product,
        output reg done
);

        wire [5:0] A2 = {A[5], A[3:0], 1'b0};
        localparam IDLE = 2'd0;
        localparam COMP = 2'd1;
        localparam DONE = 2'd2;

        reg [1:0] state, level;
        reg [5:0] partial;
        reg [1:0] prod_low;
        reg [1:0] prod_mid;
        reg s, h, d;
        reg [5:0] pin;

        always @(*) begin
                case (level)
                        2'd0: begin
                                h = (X[1] & ~X[0]) | X[0];
                                s = ~X[0];
                                d = X[1];
                                pin = 6'b0;
                        end
                        2'd1: begin
                                h = (X[3] & ~X[2]) | (~X[3] & X[1]) | (X[2] & ~X[1]);
                                s = (X[2] &  X[1]) | (~X[2] & ~X[1]);
                                d = X[3];
                                pin = {partial[5], partial[5], partial[5], partial[4], partial[3], partial[2]};
                        end
                        2'd2: begin
                                h = (X[5] & ~X[4]) | (~X[5] & X[3]) | (X[4] & ~X[3]);
                                s = (X[4] &  X[3]) | (~X[4] & ~X[3]);
                                d = X[5];
                                pin = {partial[5], partial[5], partial[5], partial[4], partial[3], partial[2]};
                        end
                        default: begin
                                h = 1'b0;
                                s = 1'b0;
                                d = 1'b0;
                                pin = 6'b0;
                        end
                endcase
        end
```

```
wire [5:0] t_out;
wire co1, co2, co3, co4, co5, co6;

SB sb5 (pin[5], A[5], A2[5], s, h, d, co5, t_out[5], co6);
SB sb4 (pin[4], A[4], A2[4], s, h, d, co4, t_out[4], co5);
SB sb3 (pin[3], A[3], A2[3], s, h, d, co3, t_out[3], co4);
SB sb2 (pin[2], A[2], A2[2], s, h, d, co2, t_out[2], co3);
SB sb1 (pin[1], A[1], A2[1], s, h, d, co1, t_out[1], co2);
SB sb0 (pin[0], A[0], A2[0], s, h, d, d, t_out[0], co1);


always @(posedge clk or posedge rst) begin
	if (rst) begin
		state <= IDLE;
		level <= 2'd0;
		done <= 1'b0;
		product <= 10'd0;
		partial <= 6'd0;
		prod_low <= 2'd0;
		prod_mid <= 2'd0;
	end
	else begin
		case (state)

			IDLE: begin
				done <= 1'b0;
				if (start) begin
					level <= 2'd0;
					state <= COMP;
				end
			end

			COMP: begin
				partial <= t_out;
				case (level)
					2'd0: prod_low <= t_out[1:0];  // keep t1[1:0]
					2'd1: prod_mid <= t_out[1:0];  // keep t2[1:0]
					2'd2: begin
						product <= {t_out, prod_mid, prod_low};
						state <= DONE;
					end
					default: ;
				endcase
				if (level != 2'd2) level <= level + 2'd1;
			end

			DONE: begin
				done <= 1'b1;
				state <= IDLE;
			end

			default: state <= IDLE;
		endcase
	end
```

```
	end
endmodule

module SB(
	input pin, x1, x2, s, h, d, cin,
	output pout, cout
);
	wire a;
	assign a = s ? x2: x1;
	assign pout = ((a ^d)&h) ^ pin ^ (cin & h);
	assign cout = (((a ^ d) & h) & (cin & h)) | (((a ^ d) & h) & pin) | pin & (cin & h);
endmodule
```

TESTBENCH

```
module testbench();
	reg clk, rst, start;
	reg signed [5:0] x,y;
	wire signed [9:0] prod;
	wire done;
	reg signed [9:0] expected;

	Booth_Mul_4 dut(.clk(clk),
			.rst(rst),
			.start(start),
			.X(x),
			.A(y),
			.product(prod),
			.done(done)
	);

	always #5 clk = ~clk;

	task multiply;
		input signed [5:0] xin, yin;
		begin
			@(negedge clk);
			x = xin;
			y = yin;
			start = 1;
			@(negedge clk);
			start = 0;
			@(posedge done);
		end
	endtask

	initial begin
		$display("Start of the test Booth multiplier");
		clk = 0; rst = 1; start = 0; x = 0; y = 0;
		#20;
		rst = 0;
		#10;
```

```
		for (x = -5; x <= 5; x = x+1) begin
			for (y = -5; y <= 5; y=y+1) begin
				multiply(x, y);
				expected = x * y;
				if (prod !== expected) $display("Error: x=%d, y=%d, prod=%d, expected=%d", x, y, prod, expected);
				else $display("Success: x=%d, y=%d, prod=%d, expected=%d", x, y, prod, expected);
			end
		end
		$display("End of the test");
		$finish;
	end

endmodule
```

SIM RESULTS

Because it's a long output, Im just going to copy and paste rather than screenshot. I'll make it match the format though:

```
[dthornburg@linux402604 P3]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Feb 12 23:22 2026
Start of the test Booth multiplier
Success: x=      -5, y=      -5, prod=      25, expected=      25
Success: x=      -5, y=      -4, prod=      20, expected=      20
Success: x=      -5, y=      -3, prod=      15, expected=      15
Success: x=      -5, y=      -2, prod=      10, expected=      10
Success: x=      -5, y=      -1, prod=       5, expected=       5
Success: x=      -5, y=       0, prod=       0, expected=       0
Success: x=      -5, y=       1, prod=      -5, expected=      -5
Success: x=      -5, y=       2, prod=     -10, expected=     -10
Success: x=      -5, y=       3, prod=     -15, expected=     -15
Success: x=      -5, y=       4, prod=     -20, expected=     -20
Success: x=      -5, y=       5, prod=     -25, expected=     -25
Success: x=      -4, y=      -5, prod=      20, expected=      20
Success: x=      -4, y=      -4, prod=      16, expected=      16
Success: x=      -4, y=      -3, prod=      12, expected=      12
Success: x=      -4, y=      -2, prod=       8, expected=       8
Success: x=      -4, y=      -1, prod=       4, expected=       4
Success: x=      -4, y=       0, prod=       0, expected=       0
Success: x=      -4, y=       1, prod=      -4, expected=      -4
Success: x=      -4, y=       2, prod=      -8, expected=      -8
Success: x=      -4, y=       3, prod=     -12, expected=     -12
Success: x=      -4, y=       4, prod=     -16, expected=     -16
Success: x=      -4, y=       5, prod=     -20, expected=     -20
Success: x=      -3, y=      -5, prod=      15, expected=      15
Success: x=      -3, y=      -4, prod=      12, expected=      12
Success: x=      -3, y=      -3, prod=       9, expected=       9
Success: x=      -3, y=      -2, prod=       6, expected=       6
Success: x=      -3, y=      -1, prod=       3, expected=       3
Success: x=      -3, y=       0, prod=       0, expected=       0
Success: x=      -3, y=       1, prod=      -3, expected=      -3
Success: x=      -3, y=       2, prod=      -6, expected=      -6
Success: x=      -3, y=       3, prod=      -9, expected=      -9
```

```
Success: x=  -3, y=  4, prod=  -12, expected=  -12
Success: x=  -3, y=  5, prod=  -15, expected=  -15
Success: x=  -2, y=  -5, prod=  10, expected=  10
Success: x=  -2, y=  -4, prod=  8, expected=  8
Success: x=  -2, y=  -3, prod=  6, expected=  6
Success: x=  -2, y=  -2, prod=  4, expected=  4
Success: x=  -2, y=  -1, prod=  2, expected=  2
Success: x=  -2, y=  0, prod=  0, expected=  0
Success: x=  -2, y=  1, prod=  -2, expected=  -2
Success: x=  -2, y=  2, prod=  -4, expected=  -4
Success: x=  -2, y=  3, prod=  -6, expected=  -6
Success: x=  -2, y=  4, prod=  -8, expected=  -8
Success: x=  -2, y=  5, prod=  -10, expected=  -10
Success: x=  -1, y=  -5, prod=  5, expected=  5
Success: x=  -1, y=  -4, prod=  4, expected=  4
Success: x=  -1, y=  -3, prod=  3, expected=  3
Success: x=  -1, y=  -2, prod=  2, expected=  2
Success: x=  -1, y=  -1, prod=  1, expected=  1
Success: x=  -1, y=  0, prod=  0, expected=  0
Success: x=  -1, y=  1, prod=  -1, expected=  -1
Success: x=  -1, y=  2, prod=  -2, expected=  -2
Success: x=  -1, y=  3, prod=  -3, expected=  -3
Success: x=  -1, y=  4, prod=  -4, expected=  -4
Success: x=  -1, y=  5, prod=  -5, expected=  -5
Success: x=  0, y=  -5, prod=  0, expected=  0
Success: x=  0, y=  -4, prod=  0, expected=  0
Success: x=  0, y=  -3, prod=  0, expected=  0
Success: x=  0, y=  -2, prod=  0, expected=  0
Success: x=  0, y=  -1, prod=  0, expected=  0
Success: x=  0, y=  0, prod=  0, expected=  0
Success: x=  0, y=  1, prod=  0, expected=  0
Success: x=  0, y=  2, prod=  0, expected=  0
Success: x=  0, y=  3, prod=  0, expected=  0
Success: x=  0, y=  4, prod=  0, expected=  0
Success: x=  0, y=  5, prod=  0, expected=  0
Success: x=  1, y=  -5, prod=  -5, expected=  -5
Success: x=  1, y=  -4, prod=  -4, expected=  -4
Success: x=  1, y=  -3, prod=  -3, expected=  -3
Success: x=  1, y=  -2, prod=  -2, expected=  -2
Success: x=  1, y=  -1, prod=  -1, expected=  -1
Success: x=  1, y=  0, prod=  0, expected=  0
Success: x=  1, y=  1, prod=  1, expected=  1
Success: x=  1, y=  2, prod=  2, expected=  2
Success: x=  1, y=  3, prod=  3, expected=  3
Success: x=  1, y=  4, prod=  4, expected=  4
Success: x=  1, y=  5, prod=  5, expected=  5
Success: x=  2, y=  -5, prod=  -10, expected=  -10
Success: x=  2, y=  -4, prod=  -8, expected=  -8
Success: x=  2, y=  -3, prod=  -6, expected=  -6
Success: x=  2, y=  -2, prod=  -4, expected=  -4
Success: x=  2, y=  -1, prod=  -2, expected=  -2
Success: x=  2, y=  0, prod=  0, expected=  0
Success: x=  2, y=  1, prod=  2, expected=  2
Success: x=  2, y=  2, prod=  4, expected=  4
```

```
Success: x=        2, y=        3, prod=        6, expected=        6
Success: x=        2, y=        4, prod=        8, expected=        8
Success: x=        2, y=        5, prod=       10, expected=       10
Success: x=        3, y=       -5, prod=      -15, expected=      -15
Success: x=        3, y=       -4, prod=      -12, expected=      -12
Success: x=        3, y=       -3, prod=       -9, expected=       -9
Success: x=        3, y=       -2, prod=       -6, expected=       -6
Success: x=        3, y=       -1, prod=       -3, expected=       -3
Success: x=        3, y=        0, prod=        0, expected=        0
Success: x=        3, y=        1, prod=        3, expected=        3
Success: x=        3, y=        2, prod=        6, expected=        6
Success: x=        3, y=        3, prod=        9, expected=        9
Success: x=        3, y=        4, prod=       12, expected=       12
Success: x=        3, y=        5, prod=       15, expected=       15
Success: x=        4, y=       -5, prod=      -20, expected=      -20
Success: x=        4, y=       -4, prod=      -16, expected=      -16
Success: x=        4, y=       -3, prod=      -12, expected=      -12
Success: x=        4, y=       -2, prod=       -8, expected=       -8
Success: x=        4, y=       -1, prod=       -4, expected=       -4
Success: x=        4, y=        0, prod=        0, expected=        0
Success: x=        4, y=        1, prod=        4, expected=        4
Success: x=        4, y=        2, prod=        8, expected=        8
Success: x=        4, y=        3, prod=       12, expected=       12
Success: x=        4, y=        4, prod=       16, expected=       16
Success: x=        4, y=        5, prod=       20, expected=       20
Success: x=        5, y=       -5, prod=      -25, expected=      -25
Success: x=        5, y=       -4, prod=      -20, expected=      -20
Success: x=        5, y=       -3, prod=      -15, expected=      -15
Success: x=        5, y=       -2, prod=      -10, expected=      -10
Success: x=        5, y=       -1, prod=       -5, expected=       -5
Success: x=        5, y=        0, prod=        0, expected=        0
Success: x=        5, y=        1, prod=        5, expected=        5
Success: x=        5, y=        2, prod=       10, expected=       10
Success: x=        5, y=        3, prod=       15, expected=       15
Success: x=        5, y=        4, prod=       20, expected=       20
Success: x=        5, y=        5, prod=       25, expected=       25
End of the test
$finish called from file "tb.v", line 48.
$finish at simulation time              6075
         V C S   S i m u l a t i o n   R e p o r t
Time: 6075
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Thu Feb 12 23:22:48 2026
[dthornburg@linux402604 P3]$
```